HIGH PERFORMANCE COMPUTING

UNIT 5

CUDA Architecture

INTRODUCTION TO GPU ARCHITECTURE OVERVIEW

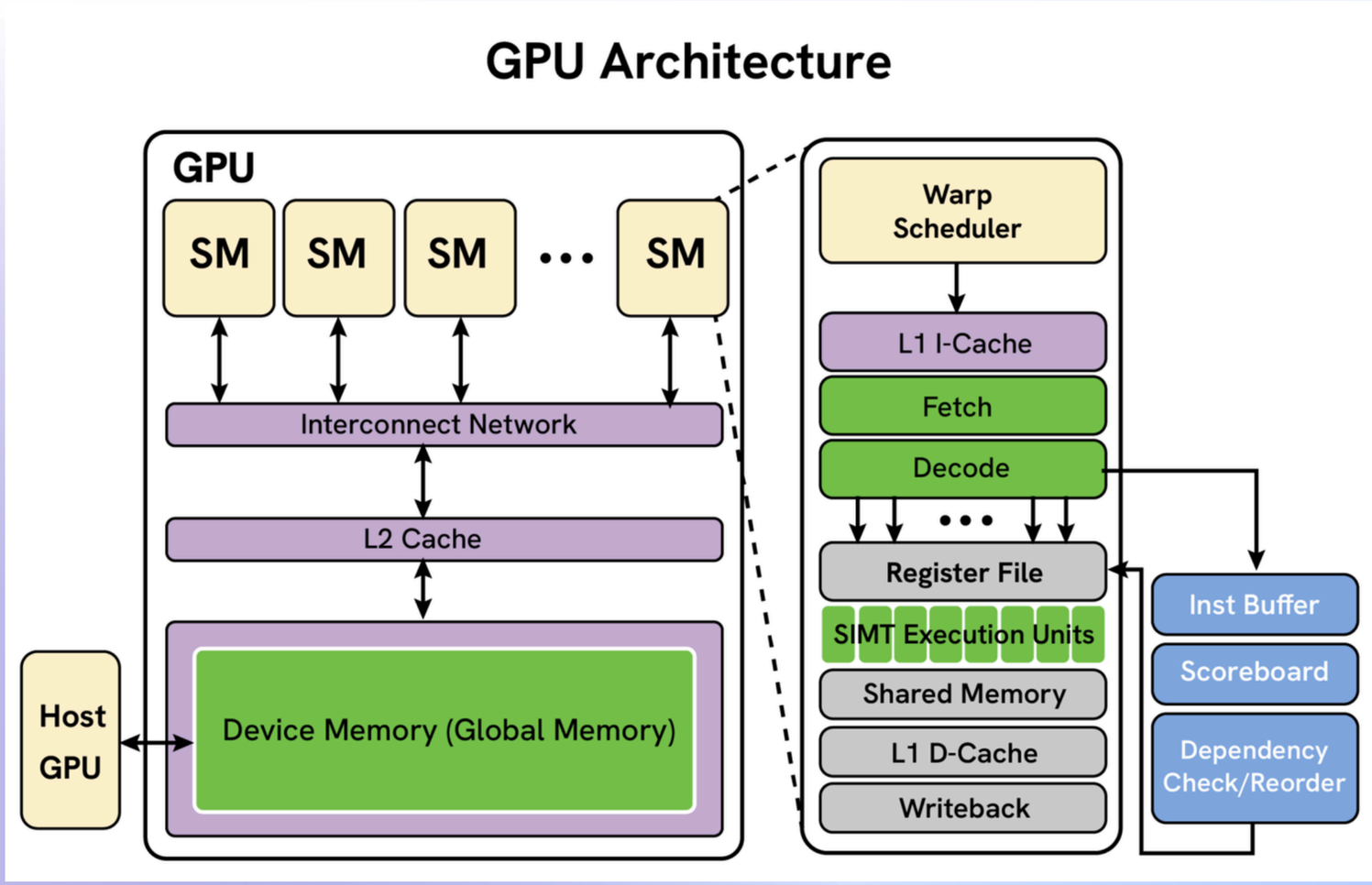
What is a GPU?

- GPU stands for Graphics Processing Unit
- Specialized processor designed for parallel processing
- Originally developed for graphics and gaming
- Now used in:
  - Artificial Intelligence (AI)
  - Machine Learning
  - Scientific Computing
  - High Performance Computing (HPC)

Key Features

- Thousands of small processing cores
- Executes multiple tasks simultaneously
- Faster for matrix and graphical computations

GPU ARCHITECTURE OVERVIEW



- GPU contains multiple Streaming Multiprocessors (SMs)
- Each SM executes thousands of threads in parallel
- Warp Scheduler manages execution of thread groups (warps)
- CUDA Cores / Execution Units perform computations
- Register File stores temporary data at high speed
- Shared Memory enables fast communication between threads
- L1 Cache & L2 Cache reduce memory access time
- Global Memory stores large datasets
- GPU follows Thread → Block → Grid hierarchy
- Designed for massive parallel processing and high performance computing (HPC)

WHAT IS CUDA?

- CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It allows developers to use the power of GPU (Graphics Processing Unit) for general purpose computing.
- CUDA enables execution of thousands of threads simultaneously, making computations much faster compared to traditional CPU-based processing for parallel tasks.

Features of CUDA

- Massive parallel processing
- High computational speed
- GPU-based execution
- Efficient memory management
- Supports scientific and AI applications

PROGRAMMING LANGUAGE SUPPORT IN CUDA

CUDA C/C++

- Most commonly used CUDA language
- Extension of C/C++
- Used to write GPU kernels and parallel programs

Features

- Direct access to GPU hardware
- High performance
- Widely used in HPC and AI

CUDA Fortran

- Fortran version supported by CUDA
- Mainly used in scientific and engineering applications

Applications

- Numerical simulations
- Weather forecasting
- Scientific computing

Python Support (PyCUDA / CUDA Libraries)

- Python can interact with CUDA using libraries like:
  - PyCUDA
  - CuPy
  - TensorFlow
  - PyTorch

Advantages

- Easy programming
- Popular in AI and Machine Learning

OpenACC and Directive-Based Languages

- Uses compiler directives for GPU acceleration
- Easier than writing full CUDA code

Benefit

- Simplifies parallel programming

APPLICATIONS OF CUDA

1. Artificial Intelligence & Deep Learning

- Training neural networks
- Image recognition
- Speech recognition
- Natural Language Processing (NLP)

2. Scientific Computing

- Weather forecasting
- Molecular simulations
- Space research
- Mathematical modeling

3. Image and Video Processing

- Video editing
- 3D rendering
- Computer vision
- Image enhancement

4. High Performance Computing (HPC)

- Matrix multiplication
- Parallel simulations
- Big data analysis
- Supercomputing applications

5. Gaming and Graphics

- Real-time graphics rendering
- Ray tracing
- Game physics calculations

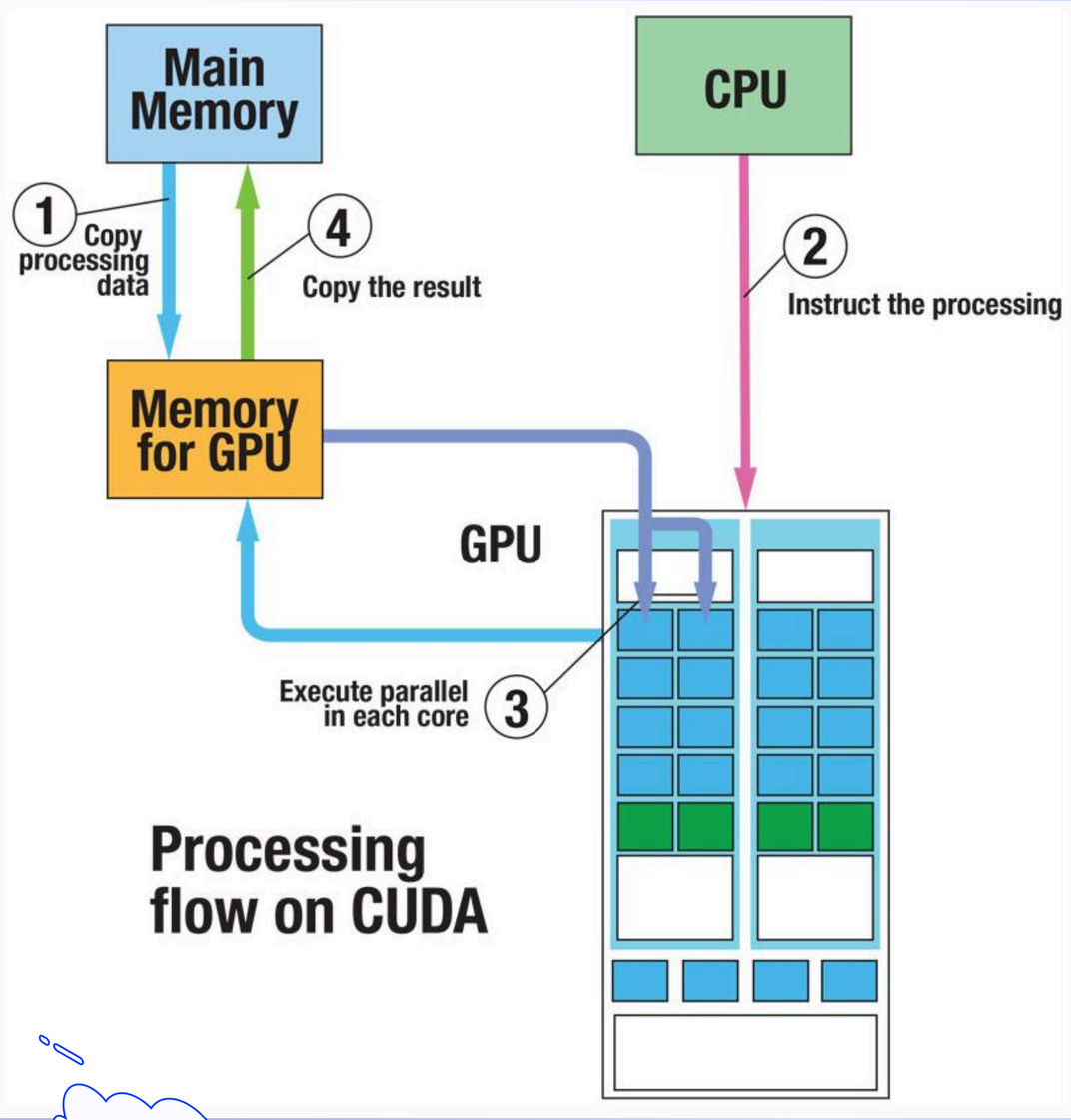
6. Data Analytics

- Large-scale data processing
- Financial analysis
- Business intelligence systems

Advantages	Explanation
High Parallel Processing	CUDA executes thousands of threads simultaneously.
Faster Computation	Performs complex calculations much faster than CPU for parallel tasks.
Efficient GPU Utilization	Uses GPU resources efficiently for maximum performance.
Scalable Performance	Performance increases with more GPU cores.
Supports Multiple Applications	Used in AI, Deep Learning, Scientific Computing, Gaming, etc.
Reduced Execution Time	Large computations complete in less time.
Easy Integration with C/C++	CUDA extends C/C++ programming language.

Disadvantages	Explanation
NVIDIA GPU Dependency	CUDA works mainly on NVIDIA GPUs only.
Complex Programming	Parallel programming is difficult for beginners.
High Power Consumption	GPUs consume more power during heavy computation.
Expensive Hardware	High-performance GPUs are costly.
Memory Transfer Overhead	Data transfer between CPU and GPU may reduce performance.
Not Suitable for Sequential Tasks	CUDA is efficient mainly for parallel workloads.
Limited Portability	CUDA programs may not run on non-NVIDIA platforms.

PROCESSING FLOW OF CUDA-C PROGRAM



1. Copy Processing Data

- Input data is copied from Main Memory to GPU Memory
- Done using cudaMemcpy()
- Purpose
- To transfer data from CPU to GPU for processing.

2. CPU Instructs the Processing

- CPU launches the CUDA kernel on GPU
- GPU receives instructions for execution
- Example
- kernel<<<Grid, Block>>>();

3. Parallel Execution on GPU

- GPU executes thousands of threads simultaneously
- Processing is done in parallel on multiple GPU cores
- Advantages
- Faster computation
- High performance
- Efficient parallel processing

4. Copy the Result Back

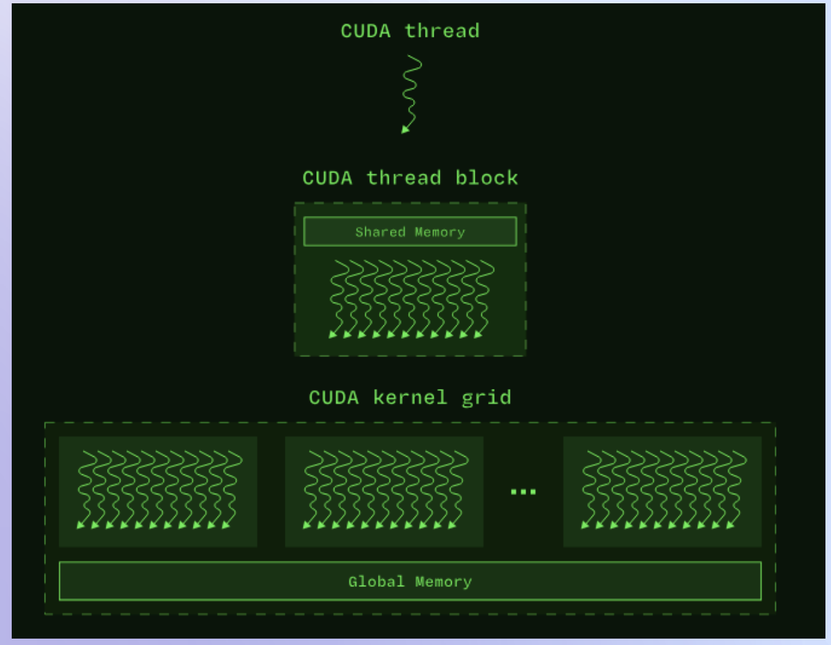
- Processed result is copied from GPU Memory to Main Memory
- Again performed using cudaMemcpy()

CUDA PROGRAMMING MODEL

The CUDA Programming Model is a parallel programming model developed by NVIDIA that allows execution of thousands of threads simultaneously on the GPU.

It organizes parallel execution using:

- Threads
- Blocks
- Grids



1. Thread

- Smallest execution unit in CUDA
- Each thread performs one task

Example

One thread adds one vector element.

2. Block

- Group of threads
- Threads inside a block can:
  - Share data
  - Synchronize execution

Block Dimension

Defines number of threads in a block.

Example:

dim3 blockDim(256);

Meaning:

- 256 threads per block

3. Grid

- Collection of blocks
- All blocks execute the same kernel

Grid Dimension

Defines number of blocks in a grid.

Example:

dim3 gridDim(16);

Meaning:

- 16 blocks in the grid

WRITE AND LAUNCH A CUDA KERNEL

- A CUDA Kernel is a special function written in CUDA-C that runs on the GPU (Device).
- The kernel is executed by many parallel threads simultaneously, which enables high-speed parallel computation.
- CUDA kernels are launched from the CPU (Host) and executed on the GPU (Device).

STEPS TO WRITE AND LAUNCH A CUDA KERNEL

1. Write the Kernel Function

A kernel function is declared using the __global__ keyword.
Example

```
__global__ void add(int *a, int *b, int *c)
{
    int i = threadIdx.x;
    c[i] = a[i] + b[i];
}
```

2. Allocate Memory on GPU

Memory is allocated on the GPU using cudaMalloc().
Example

```
cudaMalloc((void**)&d_a, size);
cudaMalloc((void**)&d_b, size);
cudaMalloc((void**)&d_c, size);
```

Purpose
Creates memory space on GPU device memory.

3. Copy Data from CPU to GPU

Input data is transferred from Host memory to Device memory using cudaMemcpy().
Example
cudaMemcpy(d_a, h_a, size,
cudaMemcpyHostToDevice);

Meaning
- h_a, h_b → Host arrays
- d_a, d_b → Device arrays

4. Launch the CUDA Kernel

Kernel launch syntax:

kernel<<<Number_of_Blocks, Threads_per_Block>>>();

Example

add<<<1, 5>>>(d_a, d_b, d_c);

5. Copy Result Back to CPU

After execution, results are copied back to Host memory.

Example

cudaMemcpy(h_c,d_c,size, cudaMemcpyDeviceToHost);

6. Free GPU Memory

Allocated GPU memory is released using cudaFree().
Example

cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

CUDA KERNEL FUNCTION TO MULTIPLY TWO INTEGERS

CUDA Kernel Program

```
__global__ void multiply(int *a, int *b, int *c)
{
    *c = (*a) * (*b);
}
```

Kernel Launch

```
multiply<<<1,1>>>(d_a, d_b, d_c);
```

Explanation

- <<<1,1>>>
- 1 Block
- 1 Thread
- GPU multiplies two integers:
- c = a × b

Working Process

- CPU sends integers to GPU memory
- Kernel is launched
- GPU thread performs multiplication
- Result stored in output variable
- Result copied back to CPU

CUDA KERNEL FOR ADDITION OF TWO VECTORS USING THREADS

CUDA Kernel Program

```
__global__ void vectorAdd(int *A, int *B, int *C)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    C[i] = A[i] + B[i];
}
```

Kernel Launch

- vectorAdd<<<2,4>>>(A, B, C);

Meaning

- 2 → Number of blocks
- 4 → Threads per block

Total Threads:

- 2 × 4 = 8 threads

How Addition is Performed Using Threads

- Each thread performs addition of one vector element independently.
- Thread Index Formula

$$i = blockIdx.x \times blockDim.x + threadIdx.x$$

Example

- A = [1 2 3 4]
- B = [5 6 7 8]

Thread	Operation	Result
Thread 0	C[0] = 1 + 5	6
Thread 1	C[1] = 2 + 6	8
Thread 2	C[2] = 3 + 7	10
Thread 3	C[3] = 4 + 8	12

Output: C = [6 8 10 12]

CUDA TERMINOLOGIES

Term	Explanation
Host	The Host refers to the CPU and its memory. It controls the execution of the CUDA program and launches kernels on the GPU.
Device	The Device refers to the GPU and its memory where parallel computation is performed.
Device Code	Device code is the part of the program that executes on the GPU (Device). It usually contains kernel functions and parallel operations.
Kernel	A Kernel is a special CUDA function executed on the GPU by many threads in parallel. It is declared using the __global__ keyword.

CUDA MEMORY MODEL

- CUDA memory model defines different types of memory used in GPU architecture for storing and accessing data during parallel execution.
- Each memory type has different speed, size, and accessibility.

Memory Type	Location	Speed	Accessibility	Purpose
Registers	Inside SM	Fastest	Single Thread	Stores temporary variables
Local Memory	Device Memory	Slow	Single Thread	Used when registers are insufficient
Shared Memory	Inside SM	Very Fast	Threads in Same Block	Data sharing among threads
Global Memory	GPU Device Memory	Slow	All Threads & CPU	Stores large data
Constant Memory	Device Memory	Fast	Read Only	Stores constant values
Texture Memory	Device Memory	Fast	Read Only	Optimized for image processing

CUDA MEMORY HIERARCHY

REGISTERS (FASTEST)

SHARED MEMORY

L1 / L2 CACHE

GLOBAL MEMORY (SLOW)

MANAGE COMMUNICATION AND SYNCHRONIZATION IN CUDA

- Communication and synchronization are important in CUDA programming because multiple threads execute in parallel on the GPU.
- Proper communication and synchronization ensure correct data sharing and accurate execution.

1. Communication in CUDA

Communication means sharing data between:

- CPU and GPU
- Threads inside GPU
- Types of Communication

a) Host to Device Communication

- Data is transferred from CPU memory to GPU memory.
- Performed using cudaMemcpy().

Example

- cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);

b) Device to Host Communication

- Result data is transferred back from GPU to CPU.

Example

- cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
-

c) Thread Communication

Threads inside the same block communicate using Shared Memory.

Advantages

- Faster communication
- Reduces global memory access

MANAGE COMMUNICATION AND SYNCHRONIZATION IN CUDA

2. Synchronization in CUDA

- Synchronization ensures that threads execute in proper order and complete tasks correctly.
- Thread Synchronization

CUDA provides synchronization function:

- __syncthreads();

Working of Synchronization

Suppose:

- Some threads are writing data into shared memory
- Other threads need that data

Without synchronization:

- Incorrect or incomplete data may be used

Using __syncthreads():

- Ensures all threads complete writing before reading starts

Example

```
shared[threadIdx.x] = data;

__syncthreads();

result = shared[threadIdx.x];
```

Explanation

- Threads store data in shared memory
- __syncthreads() waits for all threads
- Threads safely access updated data

ADVANTAGES

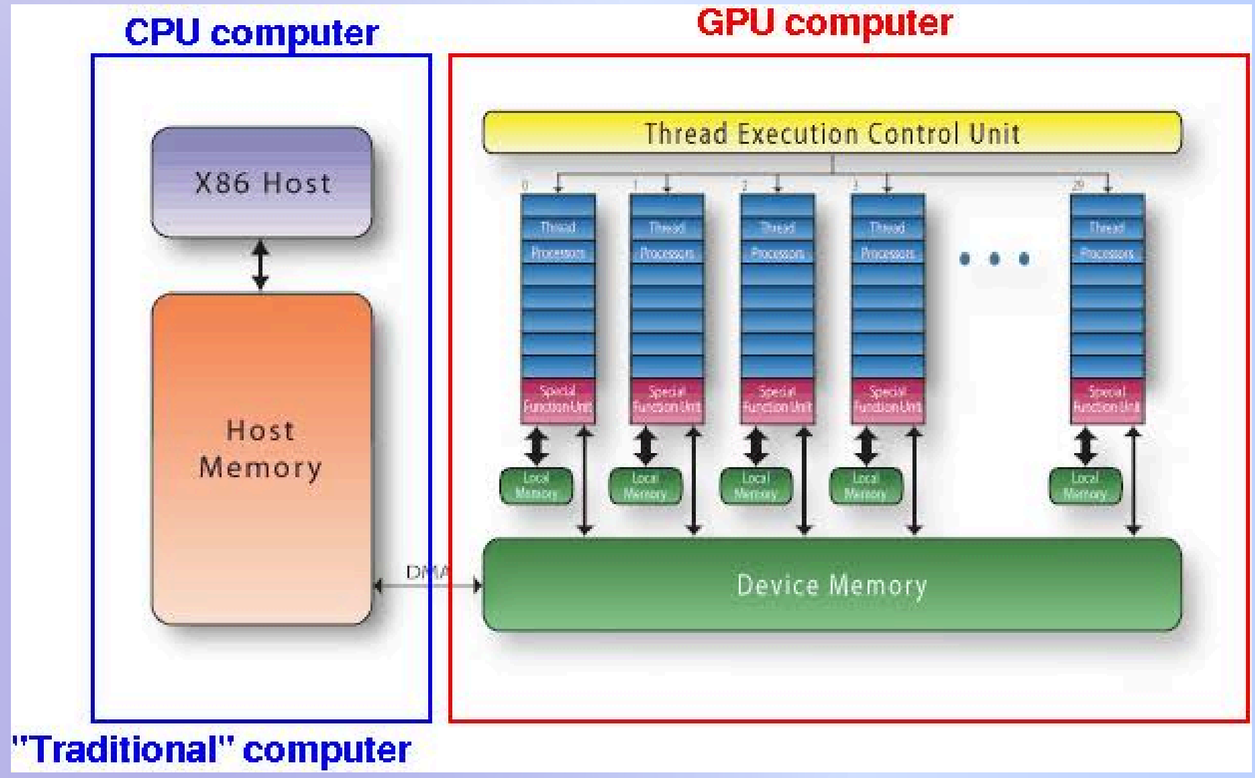
Feature	Benefit
Shared Memory Communication	Faster data sharing
Synchronization	Prevents data conflicts
Parallel Coordination	Improves execution accuracy
Efficient Thread Management	Better GPU performance

DISADVANTAGES

Limitation	Explanation
Synchronization Overhead	Waiting threads reduce speed
Complex Programming	Difficult to manage many threads
Limited Shared Memory	Small memory size per block

PARALLEL PROGRAMMING IN CUDA-C

- Parallel programming in CUDA-C allows execution of many tasks simultaneously on the GPU using thousands of threads. CUDA-C uses the parallel processing capability of GPU to improve computational speed and performance.
- CUDA follows the SIMT (Single Instruction Multiple Threads) model, where multiple threads execute the same instruction on different data.



Components in Diagram

X86 Host

- Represents CPU
- Controls program execution

Host Memory

- Stores data and instructions for CPU

Thread Execution Control Unit

- Manages execution of GPU threads

Thread Processors

- Perform parallel computations simultaneously

Special Function Unit (SFU)

- Executes complex mathematical functions

Local Memory

- Stores temporary thread data

Device Memory

- Main GPU memory used for computations

CPU	**GPU**
Sequential Processing	Parallel Processing
Few Cores	Thousands of Cores
Lower Throughput	High Throughput
Better for Sequential Tasks	Better for Large Parallel Tasks

Difference Between CUDA GPU and Traditional CPU Computing

Feature	CUDA GPU Computing	Traditional CPU Computing
Processing Style	Parallel Processing	Sequential Processing
Number of Cores	Thousands of smaller cores	Few powerful cores
Execution	Multiple tasks simultaneously	One/few tasks at a time
Performance	High for parallel tasks	Better for sequential tasks
Best Applications	AI, HPC, Graphics	General-purpose computing
Throughput	Very High	Moderate
Power Efficiency	Efficient for parallel workloads	Less efficient for massive parallelism

THANK YOU!